

Dockerfile Best Practices

Lean Images · Layer Caching · Security Hygiene

■ Analogy

- Writing a Dockerfile is like packing a suitcase. Put rarely-changed items (base OS, dependencies) at the bottom, frequently-changed items (your code) at the top. This way you don't re-fold everything when you swap one shirt — Docker reuses cached layers the same way.

Key Terms

Layer Cache	Docker caches each layer. A cache miss invalidates all subsequent layers — order matters
Build Context	Directory sent to daemon on 'docker build'. Keep it small with .dockerignore
.dockerignore	Excludes files from build context — like .gitignore for Docker
RUN	Executes a command and commits the result as a new layer
COPY vs ADD	COPY just copies files. ADD also extracts tarballs and fetches URLs — prefer COPY
ENTRYPOINT	Fixed executable that always runs (e.g., java). CMD provides default arguments
CMD	Default command/args — overridable at docker run time
USER	Sets the process user — always use a non-root user in production
ARG	Build-time variable (not available at runtime). Use for versions, tokens
HEALTHCHECK	Tells Docker how to test if the container is healthy

Good vs Bad Dockerfile

■ Bad — bloated, slow, insecure

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y openjdk-17-jdk curl wget git
COPY . /app
RUN cd /app && mvn clean package
EXPOSE 8080
CMD ["java", "-jar", "/app/target/app.jar"]
# Problems: runs as root, copies entire source, installs dev tools
```

■ Good — lean, cached, secure

```
FROM eclipse-temurin:17-jre-alpine AS runtime
# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
# Copy only the built jar (source NOT included)
COPY --chown=appuser:appgroup target/app.jar /app/app.jar
USER appuser
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java"]
CMD ["-Xmx512m", "-jar", "/app/app.jar"]
```

Layer Caching Rules

Rule	Why It Matters
Put COPY of pom.xml + RUN mvn dependency:go-offline BEFORE COPY of source	Dependencies change rarely — cache them separately
Combine RUN commands with && to reduce layers	Fewer layers = smaller image
Use --no-cache on final builds in CI	Ensures fresh security patches
Use .dockerignore to exclude target/, .git/, node_modules/	Speeds up context transfer
Pin base image versions (not 'latest')	Reproducible builds, no surprise breakages

■ Real-World: Uber

- Uber reduced their Java service Docker image sizes from 800MB to 120MB by switching to JRE-only alpine bases, separating dependency resolution from code copy, and using .dockerignore. Build times dropped from 8 minutes to under 90 seconds.